Spark con RDDs (parte 1)

Sistemas de Big Data

Sistemas de Big_Data

Curso Especialización Inteligencia_Artificial y Big_Data

EMPEZANDO UN ALGORITMO

- Necesitamos tener:
  - sparkSession
  - sparkContext
- Depende de la herramienta que usemos o de la forma de lanzar al algoritmo tendremos que crear estas variables de una forma u otra.
- En pyspark la herramienta nos da una variable sc para que accedamos al sparkContext

EMPEZANDO UN ALGORITMO

- En Jupyter tenemos que crear nuestra Sesión de Spark y a partir de la sesión el SparkContext.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()

# Creamos el spark context para poder crear RDD
sc = spark.sparkContext
```

CREANDO UN RDD

- RDD vacío

```
rddVacio1 = sc.emptyRDD()
```

- RDD un RDD a partir de una lista vacía, y le indicamos que va a tener 3 particiones.

```
rddVacio2 = sc.parallelize([],3)
```

- Comprobamos el número de particiones.

```
rddVacio2.getNumPartitions()
```

PRIMERAS OPERACIONES CON UN RDD

- Creamos un RDD con datos y sin asignarle particiones manualmente

```
rdd = sc.parallelize([1,2,3,4,5])
```

  - Sino le asignamos particiones spark le asigna por defecto las que considere mejor. Para ello puede tener en cuenta el número de núcleos de los ejecutores.

```
rdd.getNumPartitions()
```

- Conocer el número de particiones por defecto.

```
sc.defaultParallelism
```

- Obtener el contenido del RDD

```
rdd.collect()
```

CREAR UN RDD DESDE UN FICHERO

- Crear una lista en el que cada elemento de la lista es una línea del fichero.

```
rdd_texto = sc.textFile('/work/fichero.txt')
```

- Crear una lista, en la que el primer elemento es la ruta original del fichero, y el segundo el contenido de todo el fichero.

```
rddtextocompleto = sc.wholeTextFiles(ruta)
```

OPERACIONES

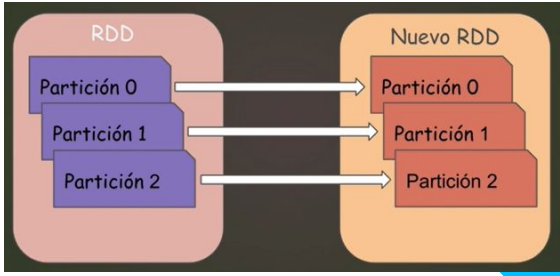
- Funciones ***lambda***:
  - Son funciones anónimas que se pueden pasar como argumento a las operaciones.
  - Una función lambda puede tener varios parámetros separados por comas.
  - En el siguiente ejemplo recibimos dos números y la función las suma.

```
lambda x,y: x+y
```

OPERACIONES

- **Map**:
  - Es una transformación por lo que no desencadena una ejecución
  - Aplica a cada elemento de un RDD la función pasada como parámetro.
  - No modifica el RDD original, sino que devuelve otro nuevo.
  - Devuelve tantos resultados de salida como tenía de entrada.
  - El resultado tiene las mismas particiones que los datos de entrada.
  - Ejemplo: sumar uno a un número dado

```
# Crear un RDD desde una lista de números
rdd = sc.parallelize([1,2,3,4,5])
# Aplicar la transformación map con una función lambda
rddsuma = rdd.map(lambda x: x+1)
rddsuma.collect()
    [2,3,4,5,6]
```



OPERACIONES

- **Filter** :
  - Es una transformación por lo que no desencadena una ejecución
  - Se utiliza para seleccionar un subconjunto de elementos de un RDD
  - Le aplica a cada elemento de un RDD la función pasada como parámetro y si esa función devuelve true, ese elemento seguirá estando en el nuevo RDD.
  - No modifica el RDD original, sino que devuelve otro nuevo.
  - Devuelve el mismo número de particiones que tenía originalmente
  - **Ejemplo** : devuelve los elementos del RDD que son distintos de 1.

```
rdd = sc.parallelize([1,2,3,4,5])
rddfiltro = rdd.filter(lambda x: x!=1)
rddfiltro.collect()
    [2,3,4,5]
```

OPERACIONES

- **Reduce**:
  - Es una operación de acción, por lo que desencadena una ejecución
  - Aplica la función pasada de manera iterativa sobre pares de datos hasta combinar todos los elementos del conjunto de datos en un sólo resultado.
  - PASOS:
    - Divide los datos en particiones dentro del clúster.
    - Aplica la función de reducción en cada partición en paralelo.
    - Combina los resultados parciales de todas las particiones hasta obtener un único valor

OPERACIONES

- **Reduce**:
  - Ejemplo:
    - Supongamos que el RDD está particionado en 2 partes:
      - Partición 1: [1, 2, 3]
      - Partición 2: [4, 5]
    - El proceso sería:
      - Aplica la función en cada partición:
        - Partición 1: ((1 + 2) + 3) = 6
        - Partición 2: (4 + 5) = 9
      - Une los resultados parciales de cada partición:
        - (6 + 9) = 15

OPERACIONES

- **Reduce**:
  - La función debe ser asociativa y conmutativa, es decir, asocia varios elementos con una operación y es conmutativa porque tiene que devolver el mismo valor sin importar el orden en el que se vayan haciendo sus operaciones
  - No devuelve un RDD, sino un resultado. En caso de operación de suma devolverá un int.
  - Para mostrar el resultado de este int:

```
print('Resultado:', valor_devuelto_por_reduce)
```

OPERACIONES

- **EJEMPLO** :

```python
# Crear un RDD desde una lista de números
rdd_ejemplo = sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Aplicar la transformación map (elevar al cuadrado cada número)
rdd_cuadrado = rdd_ejemplo.map(lambda x: x ** 2)

# Aplicar filter (quedarnos solo con los números pares)
rdd_pares = rdd_cuadrado.filter(lambda x: x % 2 == 0)

# Aplicar reduce (sumar todos los elementos del RDD filtrado)
resultado = rdd_pares.reduce(lambda x, y: x + y)

# Mostrar resultados
print("Lista original:", rdd_ejemplo.collect())
print("Lista al cuadrado:", rdd_cuadrado.collect())
print("Solo los pares:", rdd_pares.collect())
print("Suma de los pares:", resultado)
```

OPERACIONES

- **EJERCICIO** :

Realizar un programa que a partir de la siguiente lista de precios [2,5,8,13,17,20].

1.- Añade el IVA a esos precios con una función Map.

2.- Filtra los precios con IVA y nos quedamos con los que son mayores que 10.

3.- Con una función Reduce suma los valores de los precios mayores que 10.

4.- Devuelve por pantalla la suma de los precios.